PyTorch: Do Zero às GANs

Uma introdução prática para implementação de redes neurais

Material de Aula

Série de Aulas Práticas

February 25, 2026

Roteiro da Série

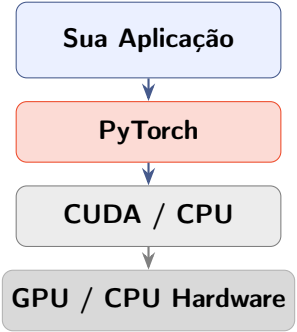
O que é PyTorch?

Definição

PyTorch é uma biblioteca Python de computação numérica acelerada por GPU com suporte nativo a **diferenciação automática**.

Por que PyTorch?

- **Define-by-run**: o grafo computacional é construído dinamicamente
- Pythônico: depuração com `print`, debugger nativo
- Dominante em pesquisa (NeurIPS, ICLR, CVPR)
- Base do Hugging Face, Lightning, JAX-like ecosystem



CUDA: CPU vs GPU

CPU

- 8–32 núcleos otimizados para *latência*
- Cache L1/L2/L3 enorme
- Bom para tarefas sequenciais
- Branches, I/O, lógica complexa

GPU

- 4.000–18.000 núcleos CUDA
- Otimizada para *throughput*
- Ideal para operações matriciais paralelas
- RTX 4090: $\approx$ 82 TFLOPS FP32

Multiplicar matrizes 1024×1024: CPU $\approx$ 500ms $\longrightarrow$ GPU $\approx$ 0.5ms. Fator $\mathbf{1000}\times$ — e redes neurais são essencialmente multiplicações de matrizes.

Identificando seu CUDA

```
# Verifica versao do driver CUDA
nvidia-smi

# Saida relevante:
# | CUDA Version: 12.8  |    <- versao do driver
#
# O driver suporta toolkits ate essa versao
# PyTorch eh compilado contra versoes especificas do toolkit
```

Versoes de Driver × Toolkit

Driver CUDA	Toolkit maximo	Wheel PyTorch
≥ 525	12.x	cu121, cu124
≥ 550	12.4+	cu124
≥ 570	12.8	cu128

Criando o Ambiente e Instalando PyTorch

```
# 1. Cria ambiente virtual isolado
python -m venv .venv
source .venv/bin/activate          # Linux/macOS
# .venv\Scripts\activate           # Windows

# 2. Instala PyTorch (ajuste cu128 para sua versao de CUDA)
pip install torch torchvision torchaudio \
    --index-url https://download.pytorch.org/whl/cu128

# 3. Verifica instalacao
python -c "
import torch
print('Versao:    ', torch.__version__)
print('CUDA disp.:', torch.cuda.is_available())
print('GPU:       ', torch.cuda.get_device_name(0))
print('Memoria:   ', torch.cuda.get_device_properties(0).total_memory // 1e9, 'GB')
"
```

Saída esperada

```
Versao:  2.5.1+cu128
CUDA disp.:  True
GPU: NVIDIA GeForce RTX 4090
Memoria:  24.0 GB
```

Seleção de Device: Padrão de Projeto

```python
import torch

# Padrao recomendado: seleciona automaticamente o melhor device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Usando: {device}")

# Verificacoes uteis durante o desenvolvimento
print(torch.cuda.device_count())                # numero de GPUs
print(torch.cuda.current_device())             # GPU ativa
torch.cuda.empty_cache()                        # libera cache VRAM
```

△ Regra de Ouro

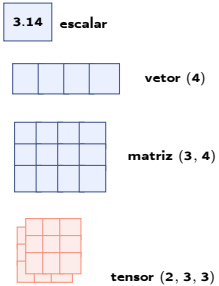
Todos os tensores e o modelo devem estar no **mesmo device**. Misturar CPU e GPU gera erro imediatamente.

O que é um Tensor?

Definição Matemática

Tensor de ordem n: elemento de $\mathbb{R}^{d_1 \times d_2 \times \cdots \times d_n}$

- **Ordem 0**: escalar — 3.14
- **Ordem 1**: vetor — shape (d)
- **Ordem 2**: matriz — shape (m, n)
- **Ordem 3**: volume — shape (C, H, W)
- **Ordem 4**: batch de imagens — shape (B, C, H, W)



Criando Tensores

```python
# A partir de dados Python
t = torch.tensor([1.0, 2.0, 3.0])

# Tensores especiais
torch.zeros(3, 4)          # zeros
torch.ones(2, 5)           # uns
torch.eye(4)               # identidade
torch.rand(3, 3)           # uniforme [0,1)
torch.randn(3, 3)          # normal N(0,1)
torch.arange(0, 10, 2)     # [0,2,4,6,8]
torch.linspace(0,1,100)    # 100 pontos
```

```python
# Inspecionando
x = torch.randn(4, 3, 32, 32)
x.shape          # torch.Size([4,3,32,32])
x.dtype          # torch.float32
x.device         # device(type='cpu')
x.ndim           # 4
x.numel()        # 4*3*32*32 = 12288

# Movendo para GPU
x_gpu = x.to(device)
x_gpu = x.cuda()       # equivalente
x_cpu = x_gpu.cpu()    # de volta
```

Tipos mais comuns

`float32` (padrão treino) · `float16` (precisão mista) · `int64` (índices) · `bool` (máscaras)

Operações Fundamentais

```python
a = torch.tensor([[1.,2.],[3.,4.]])
b = torch.tensor([[5.,6.],[7.,8.]])

# Aritmetica elemento-wise
a + b            # adicao
a * b            # multiplicacao elem.
a ** 2           # potencia

# Algebra linear
a @ b            # multiplicacao matricial
a.T              # transposta
torch.linalg.det(a)    # determinante

# Reducoes
a.sum()          # soma total
a.mean(dim=0)    # media por coluna
a.max()          # maximo
a.argmax()       # indice do maximo
```

```python
# Reshape e manipulacao
x = torch.randn(4, 3, 8, 8)
x.view(4, -1)        # (4, 192) -- flat
x.reshape(4, -1)     # igual mas + seguro
x.permute(0,2,3,1)   # (4,8,8,3)
x.squeeze()          # remove dims de 1
x.unsqueeze(0)       # adiciona dim

# Indexacao -- igual a NumPy
x[0]                 # primeiro do batch
x[:, 0]              # canal 0 de todos
x[0, :, :4, :4]    # recorte espacial

# Concatenacao
torch.cat([a, b], dim=0)    # empilha
torch.stack([a, b], dim=0)  # nova dim
```

Broadcasting — Regra Fundamental

Regra

Dois tensores são compatíveis se, para cada dimensão (alinhando pela direita), os tamanhos são **iguais** ou um deles é **1**.

```
# Exemplo pratico: normalizar batch de imagens
batch  = torch.randn(32, 3, 64, 64)   # (B, C, H, W)
mean   = torch.tensor([0.5, 0.5, 0.5])  # (3,)

# mean precisa de shape (1, 3, 1, 1) para fazer broadcast
batch_norm = batch - mean[None, :, None, None]
#              (32,3,64,64) - (1,3,1,1) -> (32,3,64,64) OK!

# Outro exemplo: adicionar bias a feature maps
feat = torch.randn(8, 128, 16, 16)   # (B, C, H, W)
bias = torch.randn(128)              # (C,)
out  = feat + bias[None, :, None, None]  # broadcast correto
```

△ Erro comum

Esquecer de ajustar as dimensoes antes do broadcast causa `RuntimeError: The size of tensor a (64) must match...`

In-place vs Out-of-place

```python
x = torch.tensor([1., 2., 3.])

# Out-of-place: cria novo tensor
y = x + 1       # x nao muda
y = x.relu()    # x nao muda

# In-place: modifica o proprio tensor
x.add_(1)           # x = [2, 3, 4]
x.relu_()
x.fill_(0.0)
# operacoes in-place tem sufixo _
```

Cuidado com in-place e autograd

Operações in-place em tensores que fazem parte do grafo computacional **destroem informações necessárias** para o backward e geram erros.

Regra prática: **evite** _ em tensores com `requires_grad=True`.

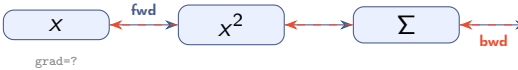
Numpy ↔ Tensor (CPU only)

```python
arr = x.numpy()           # tensor -> numpy (compartilha memoria!)
t   = torch.from_numpy(arr)    # numpy -> tensor
```

Diferenciacao Automatica

Ideia Central

Cada operacao sobre tensores com `requires_grad=True` e registrada num **grafo computacional**. `.backward()` percorre esse grafo de tras para frente calculando gradientes pela regra da cadeia.



Regra da Cadeia:

$$\frac{dL}{dx} = \frac{dL}{dy} \cdot \frac{dy}{dx}$$

Se $y = f(x)$ e $L = g(y)$:

$$\frac{dL}{dx} = g'(f(x)) \cdot f'(x)$$

Usando o Autograd

```python
import torch

# requires_grad=True: habilita rastreamento de gradientes
x = torch.tensor([2.0, 3.0], requires_grad=True)

# Operacoes sao registradas no grafo
y = x ** 2              # y = [4, 9]
L = y.sum()             # L = 13  (escalar)

# backward() calcula dL/dx para todos os nos do grafo
L.backward()

print(x.grad)
# tensor([4., 6.])    -- correto! dL/dx = 2x

# ATENCAO: gradientes acumulam! Sempre zere antes:
x.grad.zero_()
```

Por que os gradientes acumulam?

Por design — útil para gradient accumulation (simular batches maiores). Em treino normal, sempre use `optimizer.zero_grad()`.

Desabilitando o Autograd

```python
# Contexto: sem rastreamento
with torch.no_grad():
    pred = model(x_test)
    # operacoes nao vao para o grafo
    # mais rapido, menos memoria

# Decorador para funcoes inteiras
@torch.no_grad()
def avaliar(model, loader):
    for x, y in loader:
        pred = model(x)
        ...
```

```python
# .detach(): cria copia sem historico
y = model(x)
y_detached = y.detach()
# y_detached nao propaga gradientes
# util para targets em GANs

# .item(): tensor escalar -> float
loss_val = loss.item()
# IMPORTANTE: usar .item() para logs
# Chamar loss.numpy() nao zera o grafo
```

Regra prática: use `torch.no_grad()` em *qualquer* forward pass fora do treino (validação, inferência, geração). Economiza memória e acelera.

O Módulo Base: nn.Module

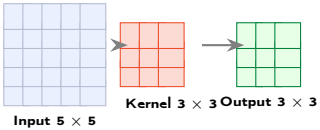
```python
import torch.nn as nn

class MinhaRede(nn.Module):
    def __init__(self):
        super().__init__()
        # Camadas declaradas aqui
        # sao REGISTRADAS automaticamente
        self.fc1 = nn.Linear(784, 256)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        # Define o forward pass
        # Backward e calculado automaticamente
        h = torch.relu(self.fc1(x))
        return self.fc2(h)

modelo = MinhaRede()
```

nn.Module gerencia:

- Parametros treinaveis (`.parameters()`)
- Sub-modulos recursivos
- Modo treino/avaliacao (`.train()`, `.eval()`)
- Serializacao (`.state_dict()`)
- Movimentacao de device (`.to(device)`)

Camadas Essenciais I — Linear e Convolucoes

```python
# Camada totalmente conectada
# y = xW^T + b
nn.Linear(in_features=512,
          out_features=256)

# Convolucao 2D
# (B,C_in,H,W) -> (B,C_out,H',W')
nn.Conv2d(in_channels=3,
          out_channels=64,
          kernel_size=3,
          padding=1,    # preserva HxW
          stride=1)

# Downsampling: stride=2 divide H,W
nn.Conv2d(64, 128, 3, stride=2, padding=1)
# Upsampling: ConvTranspose2D
nn.ConvTranspose2d(128, 64, 4,
                   stride=2, padding=1)
```



$$\text{out_size} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

Camadas Essenciais II — Normalização

```python
# BatchNorm: normaliza por batch
# Bom para batches grandes (>= 32)
nn.BatchNorm2d(num_features=64)

# GroupNorm: normaliza por grupo de canais
# Funciona com batch size pequeno
# Preferido em score matching e difusao
nn.GroupNorm(num_groups=8,
             num_channels=64)
# Regra: num_channels % num_groups == 0

# LayerNorm: normaliza por token (Transformers)
nn.LayerNorm(normalized_shape=512)

# InstanceNorm: normaliza por imagem
# Comum em style transfer
nn.InstanceNorm2d(num_features=64)
```

Formula Geral de Normalizacao

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$$

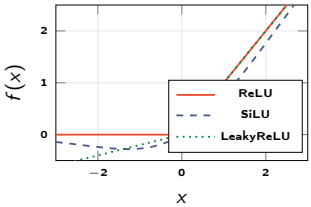
γ, β: parametros aprendiveis
ϵ: estabilidade numerica (10^{-5})

Diferenca entre elas

Tipo	Normaliza sobre
BatchNorm	batch + HW
GroupNorm	grupos de C
LayerNorm	toda a feature
InstanceNorm	H + W

Camadas Essenciais III — Ativacoes e Pooling

```python
# Ativacoes comuns
nn.ReLU()                    # max(0, x)
nn.LeakyReLU(0.2)            # usado em GANs
nn.SiLU()                    # x * sigmoid(x)
nn.GELU()                    # padrao Transformers
nn.Tanh()                    # [-1, 1] -- GANs output
nn.Sigmoid()                 # [0, 1]
nn.Softmax(dim=-1)           # soma 1 -- classificacao

# Pooling
nn.MaxPool2d(kernel_size=2, stride=2)
nn.AvgPool2d(kernel_size=2, stride=2)
nn.AdaptiveAvgPool2d((1, 1))  # global avg

# Dropout
nn.Dropout(p=0.5)            # para vetores
nn.Dropout2d(p=0.5)          # para feature maps
```



Compondo Módulos: nn.Sequential

```python
# Sequential: pilha ordenada de camadas
bloco_conv = nn.Sequential(
    nn.Conv2d(3, 64, 3, padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU(),
    nn.Conv2d(64, 64, 3, padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU(),
    nn.MaxPool2d(2, 2),
)

# ModuleList: lista de modulos
self.camadas = nn.ModuleList([
    nn.Linear(512, 256),
    nn.Linear(256, 128),
    nn.Linear(128, 64),
])

# ModuleDict: dicionario de modulos
self.heads = nn.ModuleDict({
    'classificacao': nn.Linear(64, 10),
    'regressao': nn.Linear(64, 1),
})
```

NUNCA use lista Python pura!

```python
# ERRADO -- PyTorch nao v
self.layers = [
    nn.Linear(128,64),
    nn.Linear(64, 32),
]
# parametros NAO serao treinados!

# CERTO
self.layers = nn.ModuleList([
    nn.Linear(128, 64),
    nn.Linear(64, 32),
])
```

Inspecao do Modelo

```python
model = MinhaRede().to(device)

# Conta parametros
total  = sum(p.numel() for p in model.parameters())
treina = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Total: {total:,}  |  Treinaveis: {treina:,}")

# state_dict: dicionario nome -> tensor dos parametros
for nome, param in model.state_dict().items():
    print(f"{nome:30s}  {str(param.shape):25s}  {param.numel():>8,}")

# Modo treino vs avaliacao
model.train()    # ativa Dropout e BatchNorm em modo treino
model.eval()     # desativa Dropout, BatchNorm usa media movel

# Congelar parametros (ex: transfer learning)
for param in model.fc1.parameters():
    param.requires_grad = False
```

A Abstração de Dados

Dataset

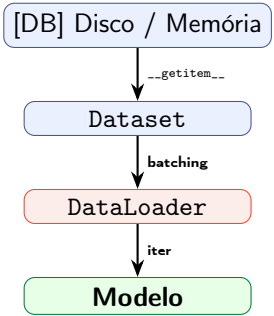
Encapsula **de onde** vêm os dados.
Dois métodos obrigatórios:

- `__len__`: tamanho total
- `__getitem__`: um item pelo índice

DataLoader

Encapsula **como** os dados chegam:

- Batching automático
- Shuffling
- Paralelismo (`num_workers`)
- Prefetching



Criando um Dataset Customizado

```python
from torch.utils.data import Dataset, DataLoader

class MeuDataset(Dataset):
    def __init__(self, imagens, rotulos, transform=None):
        # imagens: tensor (N, C, H, W) ou lista de paths
        self.imagens  = imagens
        self.rotulos  = rotulos
        self.transform = transform

    def __len__(self):
        return len(self.imagens)    # tamanho total do dataset

    def __getitem__(self, idx):
        x = self.imagens[idx]       # uma amostra
        y = self.rotulos[idx]       # seu rotulo

        if self.transform:
            x = self.transform(x)

        return x, y

# Instancia e encapsula no DataLoader
dataset = MeuDataset(X_train, y_train)
loader  = DataLoader(dataset, batch_size=64, shuffle=True, num_workers=4)
```

Usando o DataLoader

```python
# Iteracao padrao sobre um DataLoader
for x_batch, y_batch in loader:
    # x_batch: (batch_size, C, H, W)
    # y_batch: (batch_size,)
    x_batch = x_batch.to(device)
    y_batch = y_batch.to(device)
    # ... treino ...

# Datasets prontos do torchvision
import torchvision
import torchvision.transforms as T

transform = T.Compose([
    T.ToTensor(),                # PIL/numpy -> tensor [0,1]
    T.Normalize((0.5,), (0.5,)),  # normaliza para [-1,1]
    T.RandomHorizontalFlip(),  # augmentacao
])

mnist = torchvision.datasets.MNIST(
    root='./data', train=True, download=True, transform=transform
)
loader = DataLoader(mnist, batch_size=128, shuffle=True, num_workers=4)
```

Funções de Perda

Classificação

CrossEntropyLoss:

$$L = -\sum_c y_c \log \hat{p}_c$$

Combina LogSoftmax + NLLLoss.

BCEWithLogitsLoss:

$$L = -[y \log \sigma(x) + (1-y)\log(1-\sigma(x))]$$

Para classificação binária (GANs!).

Regressão

MSELoss:

$$L = \frac{1}{N}\sum(y_i - \hat{y}_i)^2$$

L1Loss (MAE):

$$L = \frac{1}{N}\sum|y_i - \hat{y}_i|$$

HuberLoss: combinação robusta de L1+L2

Usando Funções de Perda

```python
import torch.nn as nn

# CrossEntropy: logits (antes do softmax) vs indices de classe
criterio = nn.CrossEntropyLoss()
logits   = torch.randn(32, 10)    # (batch, num_classes)
rotulos  = torch.randint(0, 10, (32,))  # indices [0..9]
loss = criterio(logits, rotulos)

# BCEWithLogits: logits vs probabilidades alvo (GANs)
criterio_bce = nn.BCEWithLogitsLoss()
pred   = torch.randn(32, 1)        # discriminador
real   = torch.ones(32, 1)         # label "real" = 1
falso  = torch.zeros(32, 1)        # label "falso" = 0
loss_real  = criterio_bce(pred, real)
loss_falso = criterio_bce(pred, falso)

# MSE: para regressao e reconstrucao
criterio_mse = nn.MSELoss()
pred_reg = torch.randn(32, 1)
alvo_reg = torch.randn(32, 1)
loss_reg = criterio_mse(pred_reg, alvo_reg)
```

Otimizadores: Como os Parametros Atualizam

Regra Geral de Atualizacao

$$\theta_{t+1} = \theta_t - \alpha \cdot v_t$$

onde v_t e a **direcao** de atualizacao (depende do otimizador).

SGD com Momentum

$$v_t = \beta v_{t-1} + \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$

Acumula gradientes anteriores. Robusto, requer tuning de α.

Adam

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla L$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla L)^2$$

$$\theta \leftarrow \theta - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t$$

Step adaptativo por parametro. Padrao para deep learning.

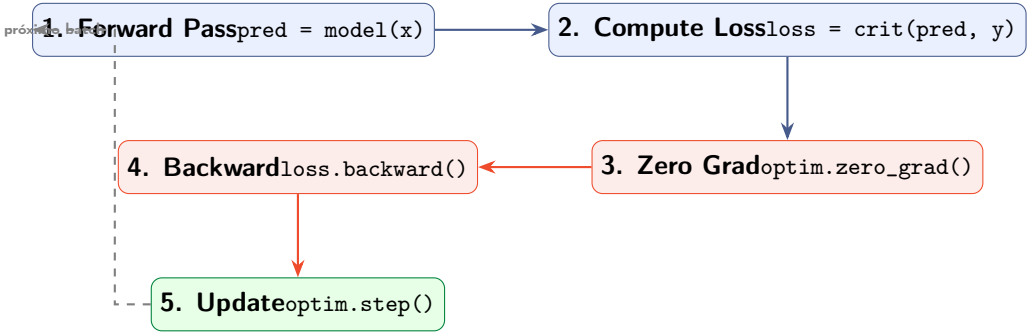
Usando Otimizadores

```python
import torch.optim as optim

model = MinhaRede().to(device)

# Adam: padrao para a maioria dos casos
optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)

# SGD com momentum: bom para CNNs supervisionadas
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

# AdamW: Adam com weight decay correto (preferido em Transformers)
optimizer = optim.AdamW(model.parameters(), lr=1e-4, weight_decay=0.01)

# Diferentes learning rates por modulo
optimizer = optim.Adam([
    {'params': model.encoder.parameters(), 'lr': 1e-4},
    {'params': model.decoder.parameters(), 'lr': 1e-3},
])

# Learning rate scheduling
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=100)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=30, gamma=0.1)
```

O Loop Completo

```python
def treinar(model, train_loader, val_loader, n_epochs=50, lr=1e-3):
    model = model.to(device)
    optimizer = optim.Adam(model.parameters(), lr=lr)
    criterio  = nn.CrossEntropyLoss()

    for epoch in range(n_epochs):
        #        FASE DE TREINO


        model.train()
        loss_treino = 0.0
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)

            pred = model(x)                     # 1. Forward
            loss = criterio(pred, y)            # 2. Calcula loss

            optimizer.zero_grad()               # 3. Zera gradientes
            loss.backward()                     # 4. Backward (gradientes)
            optimizer.step()                    # 5. Atualiza parametros

            loss_treino += loss.item()

        #        FASE DE VALIDACAO

        model.eval()
        loss_val, acertos = 0.0, 0
        with torch.no_grad():
            for x, y in val_loader:
                x, y = x.to(device), y.to(device)
                pred = model(x)
                loss_val += criterio(pred, y).item()
                acertos  += (pred.argmax(1) == y).sum().item()

        acc = acertos / len(val_loader.dataset)
```

Os 5 Passos do Loop — Mapa Mental



Ordem importa!

zero_grad() deve vir *antes* do `backward()`. Caso contrário, gradientes acumulam sobre os passos anteriores.

Salvando e Carregando Modelos

```python
#         SALVAR


# Recomendado: salva apenas os pesos (state_dict)
torch.save(model.state_dict(), 'modelo.pth')

# Checkpoint completo (para retomar treino)
torch.save({
    'epoch':      epoch,
    'model':      model.state_dict(),
    'optimizer':  optimizer.state_dict(),
    'loss':       best_loss,
}, 'checkpoint.pth')

#         CARREGAR


# Carregar pesos
model = MinhaRede()
model.load_state_dict(torch.load('modelo.pth', map_location=device))
model.eval()

# Retomar treino a partir do checkpoint
checkpoint = torch.load('checkpoint.pth')
model.load_state_dict(checkpoint['model'])
optimizer.load_state_dict(checkpoint['optimizer'])
```

Bloco Residual (ResBlock)

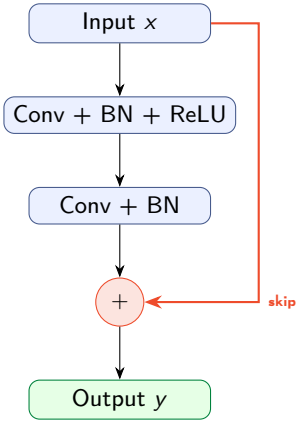
Ideia (He et al. 2015)

Em vez de aprender $H(x)$, aprende a **resíduo** $F(x) = H(x) - x$:

$$y = F(x) + x$$

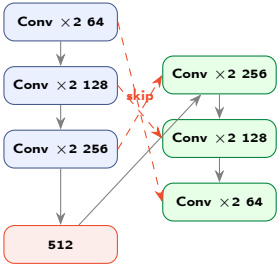
Isso facilita treinar redes muito profundas — o gradiente flui diretamente pela skip connection.

```python
class ResBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.bloco = nn.Sequential(
            nn.Conv2d(channels, channels, 3, padding=1),
            nn.BatchNorm2d(channels),
            nn.ReLU(),
            nn.Conv2d(channels, channels, 3, padding=1),
            nn.BatchNorm2d(channels),
        )
    def forward(self, x):
        return torch.relu(self.bloco(x) + x)
```



Encoder-Decoder e U-Net

```python
class UNet(nn.Module):
    def __init__(self, in_ch=1, out_ch=1):
        super().__init__()
        # Encoder: reduz espacialmente
        self.enc1 = self._bloco(in_ch, 64)
        self.enc2 = self._bloco(64, 128)
        self.enc3 = self._bloco(128, 256)
        self.pool = nn.MaxPool2d(2)

        # Bottleneck
        self.mid = self._bloco(256, 512)

        # Decoder: expande + skip connections
        self.up3 = nn.ConvTranspose2d(512,256,2,stride=2)
        self.dec3 = self._bloco(512, 256) # 256+256 skip
        self.up2 = nn.ConvTranspose2d(256,128,2,stride=2)
        self.dec2 = self._bloco(256, 128)
        self.up1 = nn.ConvTranspose2d(128, 64,2,stride=2)
        self.dec1 = self._bloco(128, 64)

        self.out = nn.Conv2d(64, out_ch, 1)

    def _bloco(self, c_in, c_out):
        return nn.Sequential(
            nn.Conv2d(c_in, c_out, 3, padding=1),
            nn.BatchNorm2d(c_out), nn.ReLU(),
            nn.Conv2d(c_out, c_out, 3, padding=1),
            nn.BatchNorm2d(c_out), nn.ReLU(),
        )

    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        e3 = self.enc3(self.pool(e2))
```



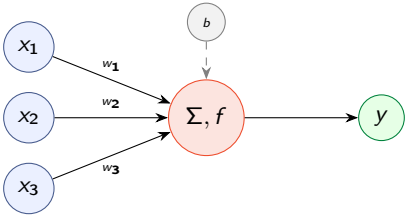
O Neuronio Artificial

Modelo de McCulloch-Pitts (1943)

O neuronio artificial recebe n entradas, calcula uma soma ponderada e aplica uma funcao de ativacao:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) = f(\mathrm{w}^\top \mathrm{x} + b)$$

Parametros aprendidos: **pesos** w e **bias** b.



- Perceptron (Rosenblatt, 1958): $f =$ step function
- Aprende com regra de Hebb: $w \leftarrow w + \eta(y - \hat{y})x$
- Limitacao: so classifica problemas **linearmente separaveis**

Perceptron em PyTorch

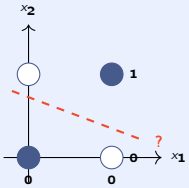
```python
import torch
import torch.nn as nn

# Um unico neuronio = nn.Linear(in, 1)
# para classificacao binaria
class Perceptron(nn.Module):
    def __init__(self, n_entradas: int):
        super().__init__()
        # w: (n_entradas,), b: escalar
        self.linear = nn.Linear(n_entradas, 1)

    def forward(self, x: torch.Tensor):
        # x: (B, n_entradas)
        logit = self.linear(x)   # (B, 1)
        return logit              # sem sigmoid
                                  # BCEWithLogits

# Treino
modelo = Perceptron(n_entradas=2)
opt    = torch.optim.SGD(
             modelo.parameters(), lr=0.1)
crit   = nn.BCEWithLogitsLoss()
```

```python
# Gerando dataset linearmente separavel
# XOR nao funcionaria com um Perceptron!
X = torch.tensor([
    [0., 0.], [0., 1.],
    [1., 0.], [1., 1.],
])
y = torch.tensor([[0.],[0.],[0.],[1.]])  # AND

# Loop simples
for epoch in range(300):
    pred = modelo(X)
    loss = crit(pred, y)
    opt.zero_grad()
    loss.backward()
    opt.step()

# Resultado
with torch.no_grad():
    prob = torch.sigmoid(modelo(X))
    print(prob.round())
    # tensor([[0.],[0.],[0.],[1.]])
```

Limitacao do Perceptron: XOR

Problema XOR

O Perceptron nao consegue aprender XOR porque XOR nao e **linearmente separavel**: nao existe uma linha reta que divida as classes.



Solucao: empilhar neuronios

Um unico neuronio so aprende fronteiras lineares.

Para aprender XOR e necessario **pelo menos uma camada oculta** com funcoes de ativacao nao-lineares — esse e o MLP.

Teorema da Aproximacao Universal: um MLP com uma camada oculta suficientemente grande pode aproximar **qualquer funcao continua**.

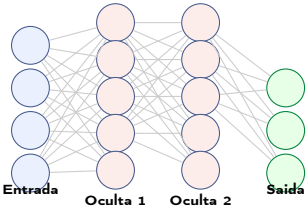
Multi-Layer Perceptron

Definicao

MLP e uma rede com pelo menos uma **camada oculta** entre entrada e saida. Cada camada aplica:

$$\mathrm{h}^{(l)} = f\left(W^{(l)}\mathrm{h}^{(l-1)} + \mathrm{b}^{(l)}\right)$$

A nao-linearidade f e essencial — sem ela, empilhar camadas seria equivalente a uma unica transformacao linear.



Por que funciona?
Cada camada aprende uma *representacao* mais abstrata da entrada. Pixels → bordas → formas → conceitos.

Implementando um MLP

```python
class MLP(nn.Module):
    """
    MLP generico e configuravel.
    hidden_dims: lista com tamanhos de cada camada oculta
    Ex: MLP(784, [512, 256, 128], 10)
    """
    def __init__(self, in_dim: int, hidden_dims: list, out_dim: int,
                 dropout: float = 0.3):
        super().__init__()

        # Constroi as camadas dinamicamente
        dims   = [in_dim] + hidden_dims
        blocos = []
        for d_in, d_out in zip(dims[:-1], dims[1:]):
            blocos += [
                nn.Linear(d_in, d_out),
                nn.BatchNorm1d(d_out),
                nn.ReLU(),
                nn.Dropout(dropout),
            ]
        self.backbone = nn.Sequential(*blocos)
        self.head     = nn.Linear(hidden_dims[-1], out_dim)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x.view(x.size(0), -1)       # achata: (B, C*H*W)
        return self.head(self.backbone(x))
```

Treinando o MLP no MNIST

```python
import torchvision, torchvision.transforms as T
from torch.utils.data import DataLoader

# Dataset
transform = T.Compose([T.ToTensor(), T.Normalize((0.5,), (0.5,))])
train_ds = torchvision.datasets.MNIST('./data', train=True,  download=True, transform=transform)
val_ds   = torchvision.datasets.MNIST('./data', train=False, download=True, transform=transform)
train_dl = DataLoader(train_ds, batch_size=256, shuffle=True,  num_workers=4)
val_dl   = DataLoader(val_ds,   batch_size=256, shuffle=False, num_workers=4)

# Modelo e otimizador
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
modelo = MLP(784, [512, 256, 128], 10).to(device)
opt    = torch.optim.Adam(modelo.parameters(), lr=1e-3)
crit   = nn.CrossEntropyLoss()

# Loop
for epoch in range(10):
    modelo.train()
    for x, y in train_dl:
        x, y = x.to(device), y.to(device)
        loss = crit(modelo(x), y)
        opt.zero_grad(); loss.backward(); opt.step()

    modelo.eval()
    acertos = sum((modelo(x.to(device)).argmax(1) == y.to(device)).sum().item()
                  for x, y in val_dl)
    print(f"Epoch {epoch+1:2d} | Acc: {acertos/len(val_ds):.4f}")
    # Esperado: ~98% apos 10 epocas
```

Problemas do MLP para Imagens

Limitacoes

- **Nao usa estrutura espacial**: pixels vizinhos sao tratados independentemente
- **Nao e invariante a translacao**: o mesmo objeto em posicoes diferentes gera ativacoes completamente distintas
- **Escala mal**: imagem $224 \times 224 \times 3 = 150.528$ entradas $\rightarrow$ primeira camada com 512 neuronios ja tem 77M parametros
- **Sem compartilhamento de parametros**: o mesmo detector de borda precisaria ser aprendido para cada posicao

Solucao: Redes Convolucionais

A CNN resolve todos esses problemas:

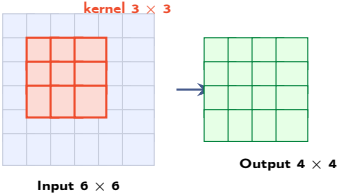
- Filtros exploram **localidade espacial**
- Filtros sao aplicados em **toda a imagem** (invariancia)
- **Compartilhamento de parametros**: um filtro 3×3 tem apenas 9 pesos independente do tamanho da imagem
- **Hierarquia**: bordas $\rightarrow$ texturas $\rightarrow$ partes $\rightarrow$ objetos

Convolucao: A Operacao Fundamental

Intuicao

Um filtro (kernel) pequeno e deslizado sobre toda a imagem. Em cada posicao, calcula o produto interno entre o filtro e o patch local da imagem — isso detecta **padroes locais**.

$$(X * W)_{h,w} = \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} X_{h+i,\, w+j} \cdot W_{i,j}$$



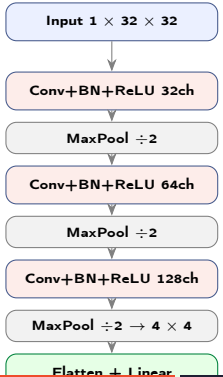
$$\text{out} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

Propriedades chave:

- **Localidade**: cada saida depende de um patch local
- **Compartilhamento**: o mesmo W para toda a imagem
- **Equivariancia**: se a entrada se desloca, a

Arquitetura CNN Classica: LeNet-like

```python
class CNN(nn.Module):
    """
    CNN para classificacao de imagens 32x32.
    Segue o padrao: Conv-BN-ReLU-Pool (encoder)
    seguido de MLP (cabeca de classificacao).
    """
    def __init__(self, num_classes: int = 10):
        super().__init__()

        # Extrator de features (encoder)
        self.features = nn.Sequential(
            # Bloco 1: (B,1,32,32) -> (B,32,16,16)
            nn.Conv2d(1, 32, 3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),

            # Bloco 2: (B,32,16,16) -> (B,64,8,8)
            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),

            # Bloco 3: (B,64,8,8) -> (B,128,4,4)
            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128), nn.ReLU(),
```



Transfer Learning com CNNs Pre-treinadas

```python
import torchvision.models as models

# Carrega ResNet-18 pre-treinada no ImageNet
backbone = models.resnet18(weights='IMAGENET1K_V1')

# Estrategia 1: Feature Extractor
# Congela TODOS os pesos, substitui apenas a cabeca
for param in backbone.parameters():
    param.requires_grad = False

# Substitui a camada final para o numero de classes desejado
backbone.fc = nn.Linear(backbone.fc.in_features, num_classes)
# So a cabeca (fc) sera treinada

# Estrategia 2: Fine-tuning
# Descongela tudo, usa LR menor para as camadas iniciais
for param in backbone.parameters():
    param.requires_grad = True

optimizer = torch.optim.Adam([
    {'params': backbone.layer4.parameters(), 'lr': 1e-4},
    {'params': backbone.fc.parameters(),     'lr': 1e-3},  # LR maior
])

# Modelos disponiveis: resnet18/50/101, efficientnet_b0/b7,
#                      vit_b_16, densenet121, mobilenet_v3_small ...
```

Dados Sequenciais e o Problema da Memoria

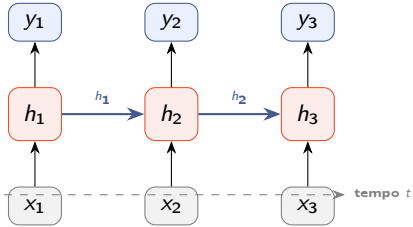
Por que RNNs?

MLPs e CNNs assumem que as entradas sao **independentes**. Em sequencias (texto, audio, series temporais), o contexto passado importa.

A RNN mantem um **estado oculto** h_t que carrega informacao dos passos anteriores:

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

$$y_t = W_y h_t + b_y$$



RNN Simples, LSTM e GRU em PyTorch

```python
# RNN simples (problema: vanishing gradient)
rnn = nn.RNN(
    input_size=64,      # dim de cada x_t
    hidden_size=128,    # dim do estado oculto
    num_layers=2,       # RNNs empilhadas
    batch_first=True,   # (B, T, F) em vez de (T,B,F)
    dropout=0.3,
)

# LSTM: gates resolvem vanishing gradient
lstm = nn.LSTM(
    input_size=64, hidden_size=128,
    num_layers=2, batch_first=True,
    dropout=0.3, bidirectional=False,
)

# GRU: mais leve que LSTM, desempenho similar
gru = nn.GRU(
    input_size=64, hidden_size=128,
    num_layers=2, batch_first=True,
)
```

```python
# Forward pass
x = torch.randn(32, 50, 64)
# x: (batch=32, seq_len=50, input=64)

# RNN/GRU: retorna (output, h_n)
out, h_n = gru(x)
# out: (32, 50, 128) -- todos os h_t
# h_n: (2, 32, 128)  -- ultimo h_t

# LSTM: retorna (output, (h_n, c_n))
out, (h_n, c_n) = lstm(x)
# c_n: cell state (especifico do LSTM)

# Classificacao de sequencia:
# usa apenas o ultimo passo de tempo
ultimo = out[:, -1, :]    # (B, 128)
logits = nn.Linear(128, num_classes)(ultimo)

# Para classificacao com LSTM
# inicializacao do estado oculto:
h0 = torch.zeros(2, 32, 128)
c0 = torch.zeros(2, 32, 128)
out, _ = lstm(x, (h0, c0))
```

Exemplo Completo: Classificador de Texto com GRU

```python
class ClassificadorGRU(nn.Module):
    """
    Classifica sequencias de tokens em num_classes categorias.
    Fluxo: indices de tokens -> embeddings -> GRU -> classificacao
    """
    def __init__(self, vocab_size: int, embed_dim: int,
                 hidden_dim: int, num_classes: int, num_layers: int = 2):
        super().__init__()
        # Embedding: converte indices inteiros em vetores densos
        self.embedding = nn.Embedding(vocab_size, embed_dim,
                                      padding_idx=0)  # 0 = <PAD>
        self.gru = nn.GRU(embed_dim, hidden_dim, num_layers,
                          batch_first=True, dropout=0.3,
                          bidirectional=True)    # bidirecional: le em ambas direcoes
        # *2 porque bidirecional concatena forward + backward
        self.head = nn.Linear(hidden_dim * 2, num_classes)
        self.dropout = nn.Dropout(0.3)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (B, T) -- indices de tokens (inteiros)
        emb = self.dropout(self.embedding(x))  # (B, T, embed_dim)
        out, _ = self.gru(emb)                 # (B, T, hidden*2)
        # Usa o ultimo token como representacao da sequencia
        return self.head(self.dropout(out[:, -1, :]))

# Uso
modelo = ClassificadorGRU(vocab_size=10000, embed_dim=128,
                          hidden_dim=256, num_classes=2)
x = torch.randint(0, 10000, (32, 100))  # batch de 32 sequencias de len 100
print(modelo(x).shape)                  # (32, 2)
```

RNN vs LSTM vs GRU: Quando usar cada uma

	RNN	**LSTM**	**GRU**
Parametros	menos	mais	medio
Vanishing gradient	sim	resolvido	resolvido
Velocidade de treino	rapida	lenta	media
Sequencias longas	ruim	otima	boa
Use quando	prototipo rapido	dependencias longas	balanco perf/custo

Atencao: Em NLP moderno, Transformers (BERT, GPT) superam RNNs em quase todas as tarefas. RNNs ainda sao relevantes para **series temporais**, **audio** e cenarios de **recursos limitados** onde a complexidade quadratica do Transformer e proibitiva.

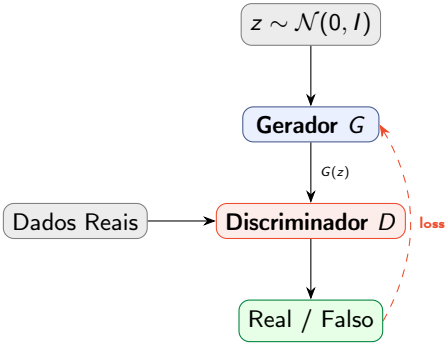
O que é uma GAN?

Ideia (Goodfellow et al. 2014)

Duas redes em **jogo adversarial**:

- **Gerador** G: transforma ruído $z \sim p_z$ em dados falsos
- **Discriminador** D: distingue dados reais de falsos

G tenta enganar D; D tenta não ser enganado.



Objetivo (minimax):

$$\min_G \max_D \; \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

Gerador e Discriminador — MLP simples

```python
# Gerador: ruido z -> imagem falsa
class Gerador(nn.Module):
    def __init__(self, z_dim=100, img_dim=784):  # 784 = 28*28 (MNIST)
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(z_dim, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, img_dim),
            nn.Tanh(),          # output em [-1, 1]
        )
    def forward(self, z):
        return self.net(z)

# Discriminador: imagem -> probabilidade de ser real
class Discriminador(nn.Module):
    def __init__(self, img_dim=784):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(img_dim, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
            # SEM Sigmoid aqui -- BCEWithLogitsLoss ja aplica
```

Loop de Treino da GAN

```
G = Gerador(z_dim=100).to(device)
D = Discriminador().to(device)
opt_G = optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))
opt_D = optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))
criterio = nn.BCEWithLogitsLoss()

for epoch in range(n_epochs):
    for x_real, _ in loader:
        x_real = x_real.view(-1, 784).to(device)  # achata para MLP
        B = x_real.size(0)

        #         TREINO DO DISCRIMINADOR

        z       = torch.randn(B, 100, device=device)
        x_falso = G(z).detach()   # .detach(): nao propaga para G!

        pred_real  = D(x_real)
        pred_falso = D(x_falso)
        loss_D = criterio(pred_real,  torch.ones_like(pred_real)) + \
                 criterio(pred_falso, torch.zeros_like(pred_falso))

        opt_D.zero_grad(); loss_D.backward(); opt_D.step()

        #         TREINO DO GERADOR

        z       = torch.randn(B, 100, device=device)
        x_falso = G(z)              # SEM .detach() -- precisamos do gradiente
        pred    = D(x_falso)
        # G quer que D classifique seus outputs como "real" (label=1)
        loss_G  = criterio(pred, torch.ones_like(pred))

        opt_G.zero_grad(); loss_G.backward(); opt_G.step()
```

O Detalhe Crucial: `.detach()`

Treino do Discriminador

Queremos atualizar **apenas** D.
Se não usarmos `.detach()`, o grafo computacional conecta D a G, e gradientes fluiriam para G indevidamente.

`x_falso = G(z).detach()`
cria uma cópia sem histórico — D vê apenas o tensor, sem saber que veio de G.

Treino do Gerador

Queremos atualizar **apenas** G através de D.
Não use `.detach()` aqui — o gradiente precisa percorrer:

$$D(G(z)) \xrightarrow{\text{backward}} G$$

Por isso usamos dois otimizadores separados e zeramos gradientes independentemente.

DCGAN: GAN com Convoluções

```python
# Gerador DCGAN (z -> imagem 64x64)
class GeradorDC(nn.Module):
    def __init__(self, z_dim=100, ngf=64):
        super().__init__()
        self.net = nn.Sequential(
            # z: (B, 100, 1, 1) -> (B, ngf*8, 4, 4)
            nn.ConvTranspose2d(z_dim, ngf*8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf*8), nn.ReLU(),
            # (B, ngf*8, 4, 4) -> (B, ngf*4, 8, 8)
            nn.ConvTranspose2d(ngf*8, ngf*4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf*4), nn.ReLU(),
            # -> (B, ngf*2, 16, 16)
            nn.ConvTranspose2d(ngf*4, ngf*2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf*2), nn.ReLU(),
            # -> (B, ngf, 32, 32)
            nn.ConvTranspose2d(ngf*2, ngf, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf), nn.ReLU(),
            # -> (B, 3, 64, 64)
            nn.ConvTranspose2d(ngf, 3, 4, 2, 1, bias=False),
            nn.Tanh(),
        )
    def forward(self, z):
        return self.net(z.view(z.size(0), -1, 1, 1))
```

Discriminador DCGAN

```python
class DiscriminadorDC(nn.Module):
    def __init__(self, ndf=64):
        super().__init__()
        self.net = nn.Sequential(
            # (B, 3, 64, 64) -> (B, ndf, 32, 32)
            nn.Conv2d(3, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2),
            # -> (B, ndf*2, 16, 16)
            nn.Conv2d(ndf, ndf*2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*2), nn.LeakyReLU(0.2),
            # -> (B, ndf*4, 8, 8)
            nn.Conv2d(ndf*2, ndf*4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*4), nn.LeakyReLU(0.2),
            # -> (B, ndf*8, 4, 4)
            nn.Conv2d(ndf*4, ndf*8, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*8), nn.LeakyReLU(0.2),
            # -> (B, 1, 1, 1)
            nn.Conv2d(ndf*8, 1, 4, 1, 0, bias=False),
        )
    def forward(self, x):
        return self.net(x).view(-1, 1)  # achata para (B, 1)
```

Inicialização de Pesos

```python
# DCGAN recomenda: N(0, 0.02) para Conv e ConvTranspose
#                  N(1, 0.02) + bias=0 para BatchNorm
def inicializar_pesos(m):
    classname = m.__class__.__name__
    if 'Conv' in classname:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif 'BatchNorm' in classname:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)

G = GeradorDC().to(device)
D = DiscriminadorDC().to(device)

G.apply(inicializar_pesos)  # aplica recursivamente a todos os submodulos
D.apply(inicializar_pesos)
```

`.apply(fn)`

Percorre recursivamente todos os sub-modulos e aplica `fn` a cada um. Essencial para inicializacoes customizadas.

Boas Praticas para GANs

✓ Faca

- Use `LeakyReLU(0.2)` no discriminador
- Use `Tanh` no output do gerador
- Normalize dados para $[-1, 1]$
- Dois otimizadores separados (G e D)
- `bias=False` com BatchNorm
- Inicialize pesos com $\mathcal{N}(0, 0.02)$
- Adam com $\beta_1 = 0.5$

× Evite

- Treinar D muito mais que G (ou vice-versa)
- Usar BatchNorm no input do D
- Usar BatchNorm no output do G
- Learning rate alto ($> 5 \times 10^{-4}$)
- Batch size muito pequeno (< 32)

Mode collapse (o gerador produz sempre a mesma saida) e o problema mais comum. Sinal: `loss_D` cai para zero, `loss_G` explode.

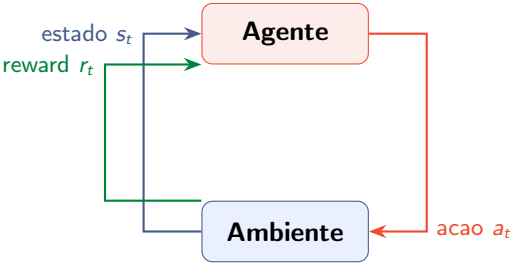
O Paradigma do Aprendizado por Reforco

Definicao

Um **agente** interage com um **ambiente** tomando **acoes**, observando **estados** e recebendo **recompensas**. O objetivo e aprender uma **politica** $\pi(a|s)$ que maximiza a recompensa acumulada futura:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

$\gamma \in (0, 1]$: fator de desconto (valoriza recompensas proximas).



Diferenca dos outros paradigmas:

- Sem rotulos supervisionados
- Feedback esparso e atrasado (credito temporal)
- Agente influencia os proprios dados que coleta

Conceitos Fundamentais

Funcao Valor $V^\pi(s)$

Recompensa acumulada esperada a partir do estado s seguindo π:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

Funcao Q $Q^\pi(s, a)$

Recompensa acumulada esperada tomando acao a em s, depois seguindo π:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$$

Politica otima: $\pi^*(s) = \arg\max_a Q^*(s, a)$

Equacao de Bellman

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

Relaciona o valor atual com o valor do proximo estado. Base de todos os algoritmos de Q-learning.

Algoritmos principais

- **DQN**: Q-learning com rede neural
- **PPO**: Policy gradient com clipping
- **A3C**: Actor-Critic assincronico
- **SAC**: Maximo de entropia (continuo)

DQN: Deep Q-Network

```python
class DQN(nn.Module):
    """
    Rede Q: mapeia estado -> Q-valor de cada acao.
    Para o CartPole: estado (4,), acoes (2,)
    """
    def __init__(self, state_dim: int,
                 n_actions: int):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, n_actions),
        )

    def forward(self, s: torch.Tensor):
        return self.net(s)
        # retorna Q(s, a) para cada acao

# Politica epsilon-greedy
def escolher_acao(model, estado, epsilon):
    if torch.rand(1).item() < epsilon:
        return torch.randint(n_actions, (1,)).
            item()
```

Replay Buffer

O DQN usa um buffer para armazenar transicoes $(s, a, r, s', done)$ e amostrar mini-batches. Isso quebra correlacao entre amostras consecutivas.

```python
from collections import deque
import random

class ReplayBuffer:
    def __init__(self, capacity=10000):
        self.buffer = deque(maxlen=capacity)

    def push(self, s, a, r, s_, done):
        self.buffer.append((s,a,r,s_,done))

    def sample(self, batch_size):
        batch = random.sample(
            self.buffer, batch_size)
        s,a,r,s_,d = zip(*batch)
        return (torch.tensor(s,  dtype=torch.float32),
                torch.tensor(a),
                torch.tensor(r,  dtype=torch.float32),
                torch.tensor(s_, dtype=torch.float32),
                torch.tensor(d,  dtype=torch.float32))
```

Loop de Treino do DQN

```python
import gymnasium as gym  # pip install gymnasium

env = gym.make('CartPole-v1')  # estado: (4,), acoes: {0, 1}

modelo     = DQN(state_dim=4, n_actions=2).to(device)
target_net = DQN(state_dim=4, n_actions=2).to(device)
target_net.load_state_dict(modelo.state_dict())   # copia inicial

buffer    = ReplayBuffer(capacity=10000)
opt       = torch.optim.Adam(modelo.parameters(), lr=1e-4)
crit      = nn.MSELoss()
epsilon   = 1.0    # exploracao inicial: 100%
gamma     = 0.99

for episodio in range(500):
    estado, _ = env.reset()
    for passo in range(500):
        acao  = escolher_acao(modelo, estado, epsilon)
        prox, reward, done, trunc, _ = env.step(acao)
        buffer.push(estado, acao, reward, prox, done or trunc)
        estado = prox

        if len(buffer) >= 512:
            s, a, r, s_, d = [t.to(device) for t in buffer.sample(64)]
            q_atual  = modelo(s).gather(1, a.unsqueeze(1)).squeeze()
            q_target = r + gamma * target_net(s_).max(1).values * (1 - d)
            loss = crit(q_atual, q_target.detach())   # .detach(): target fixo!
            opt.zero_grad(); loss.backward(); opt.step()

        if done or trunc: break

    epsilon = max(0.01, epsilon * 0.995)              # decai epsilon
    if episodio % 10 == 0:                            # atualiza rede target
        target_net.load_state_dict(modelo.state_dict())
```

REINFORCE: Policy Gradient Simples

Ideia

Em vez de aprender $Q(s, a)$, otimiza diretamente a politica $\pi_\theta(a|s)$. O gradiente da politica e:

$$\nabla_\theta J \propto \mathbb{E}[G_t \nabla_\theta \log \pi_\theta(a_t|s_t)]$$

Acoes que levaram a alta recompensa tem sua probabilidade **aumentada**.

```python
class PolicyNet(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128), nn.ReLU(),
```

```python
def treinar_episodio(env, policy, opt):
    estados, acoes, rewards = [], [], []
    s, _ = env.reset()
    done = False

    while not done:
        probs = policy(torch.tensor(s))
        dist  = torch.distributions.Categorical(probs)
        a     = dist.sample()          # amostra acao
        s, r, done, trunc, _ = env.step(a.item())
        estados.append(s); acoes.append(a)
        rewards.append(r)
        done = done or trunc

    # Retornos descontados
    G, retornos = 0, []
    for r in reversed(rewards):
        G = r + 0.99 * G
        retornos.insert(0, G)
    R = torch.tensor(retornos)
    R = (R - R.mean()) / (R.std() + 1e-8)   # baseline
```

RL no PyTorch: Resumo dos Componentes

Componente	O que e	PyTorch
Politica $\pi_\theta(a\|s)$	Rede que decide acoes	`nn.Module`
Funcao Q $Q_\theta(s,a)$	Rede que estima valores	`nn.Module`
Replay Buffer	Armazena transicoes	`deque` / `Tensor`
Target Network	Copia estavel da Q-net	`load_state_dict`
Distribuicao de acao	Amostragem diferenciavel	`torch.distributions`
Loss de valor	Bellman residual	`nn.MSELoss`
Loss de politica	$-\log \pi \cdot G$	calculo manual

Para RL em producao, use bibliotecas especializadas: **Stable-Baselines3** (PyTorch), **RLlib**, ou **CleanRL**. Elas ja implementam DQN, PPO, SAC com hiperparametros validados.

Monitorando o Treino com TensorBoard

```python
from torch.utils.tensorboard import SummaryWriter

writer = SummaryWriter('runs/experimento_01')

for epoch in range(n_epochs):
    # ... loop de treino ...

    # Escalares
    writer.add_scalar('Loss/Gerador',       loss_G.item(), epoch)
    writer.add_scalar('Loss/Discriminador', loss_D.item(), epoch)

    # Imagens geradas
    with torch.no_grad():
        z = torch.randn(64, 100, device=device)
        amostras = G(z).reshape(-1, 1, 28, 28) * 0.5 + 0.5
        writer.add_images('Amostras/Geradas', amostras, epoch)

    # Histograma de pesos
    for nome, param in G.named_parameters():
        writer.add_histogram(f'G/{nome}', param, epoch)

writer.close()
```

```bash
tensorboard --logdir=runs     # abre no navegador: http://localhost:6006
```

Depurando Problemas Comuns

Gradientes explodindo

```
# Sintoma: loss = nan, inf
# Solucao: gradient clipping
nn.utils.clip_grad_norm_(
    model.parameters(),
    max_norm=1.0
)
# Sempre ANTES do optimizer.step()
```

Sem memoria (OOM)

```
# CUDA out of memory
# 1. Reduza batch_size
# 2. Use no_grad na validacao
# 3. Libere cache
torch.cuda.empty_cache()
# 4. Acumule gradientes
loss = loss / n_acc
loss.backward()
if step % n_acc == 0:
    optimizer.step()
    optimizer.zero_grad()
```

Tensores em devices errados

```
# RuntimeError: Expected all tensors
# to be on the same device
# Solucao: mova tudo para device
x = x.to(device)
y = y.to(device)
model = model.to(device)
```

Verifica shapes

```
# Adicione prints no forward:
def forward(self, x):
    print(x.shape)    # debug!
    x = self.conv(x)
    print(x.shape)
```

Precisao Mista (Automatic Mixed Precision)

```python
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()

for x, y in loader:
    x, y = x.to(device), y.to(device)

    # autocast: usa float16 onde e seguro, float32 onde necessario
    with autocast():
        pred = model(x)
        loss = criterio(pred, y)

    optimizer.zero_grad()

    # scaler: evita underflow de gradientes em float16
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

Benefícios da Precisão Mista

- $\approx 2\times$ mais rápido em GPUs com Tensor Cores (Ampere, Ada)
- Reduz uso de VRAM pela metade para os activations

Mapa Conceitual Completo

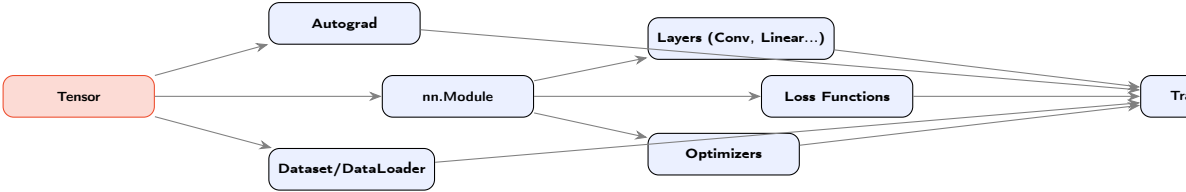


Tabela de Referência: Camadas

Camada	Input	Uso típico
`nn.Linear(in, out)`	(B, in)	MLP, head
`nn.Conv2d(in, out, k)`	(B, in, H, W)	Feature extraction
`nn.ConvTranspose2d`	(B, in, H, W)	Upsampling (GAN, U-Net)
`nn.BatchNorm2d(C)`	(B, C, H, W)	Estabilizar treino
`nn.GroupNorm(G, C)`	(B, C, H, W)	Batch size pequeno
`nn.MaxPool2d(2)`	(B, C, H, W)	Downsampling
`nn.Upsample(2)`	(B, C, H, W)	Upsampling suave
`nn.Dropout(p)`	qualquer	Regularização
`nn.Embedding(V, D)`	(B, L) int	NLP, tokens

Tabela de Referência: Os 5 Passos

Passo	Código	O que acontece
1	`pred = model(x)`	Forward pass, constrói grafo
2	`loss = crit(pred, y)`	Calcula escalar da loss
3	`optimizer.zero_grad()`	Zera gradientes acumulados
4	`loss.backward()`	Backprop, preenche `.grad`
5	`optimizer.step()`	Atualiza $\theta \leftarrow \theta - \alpha\nabla$

Este ciclo de 5 passos é a base de **todo** treinamento em PyTorch — de uma regressão linear a uma GAN com 100M de parâmetros. A diferença está apenas na complexidade do modelo e na lógica em torno do loop.

Fim da Série

Do Tensor à GAN

Documentação oficial: `pytorch.org/docs`

`pytorch.org/tutorials` · `github.com/pytorch/examples`